



Funciones



¿Podrías definir con tus palabras qué es una variable?



Recursos asincrónicos

¿Revisaste los recursos de la semana 3?

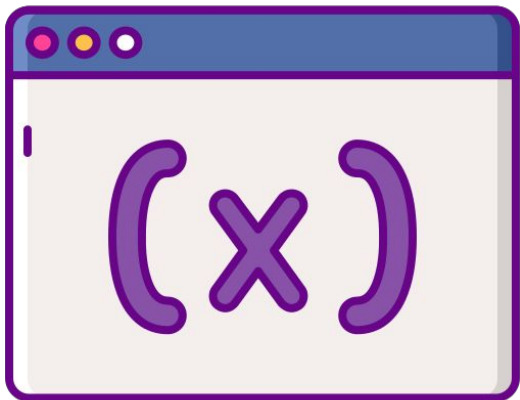
- *Presentación de clases*
- *Guía - Funciones*
- *Desafío – Funciones*

¿Tienes dudas sobre el material o algo revisado en ellos?



/*Variables*/

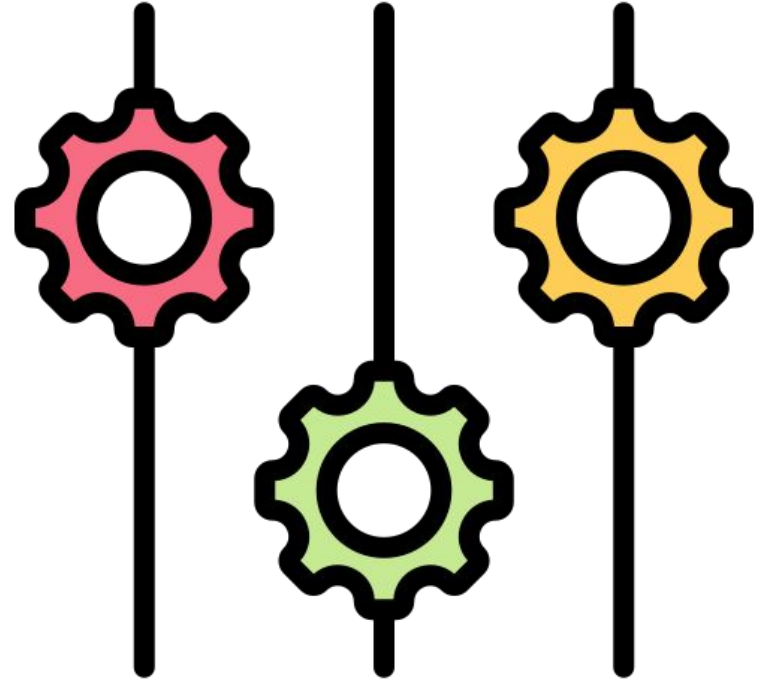
¿Qué es son las Variables en Javascript?



- Una variable en JavaScript es como una caja etiquetada donde puedes guardar y manipular información.
- Imagina una caja llamada "edad" donde puedes poner el número 25. Más tarde, puedes abrir la caja, mirar dentro y ver que la "edad" es 25.
- Las variables te permiten almacenar y trabajar con datos, como números, texto o cualquier otra cosa, haciéndolas esenciales para programar de manera dinámica y flexible.

var, let y const

- **var**: es una palabra reservada que sirve para declarar una variable cuyo alcance será global.
- **let**: similar a var, pero tiene un alcance de bloque.
- **const**: se define como constante, es una forma guardar en memoria datos cuyos valores no cambian.



/*Funciones */

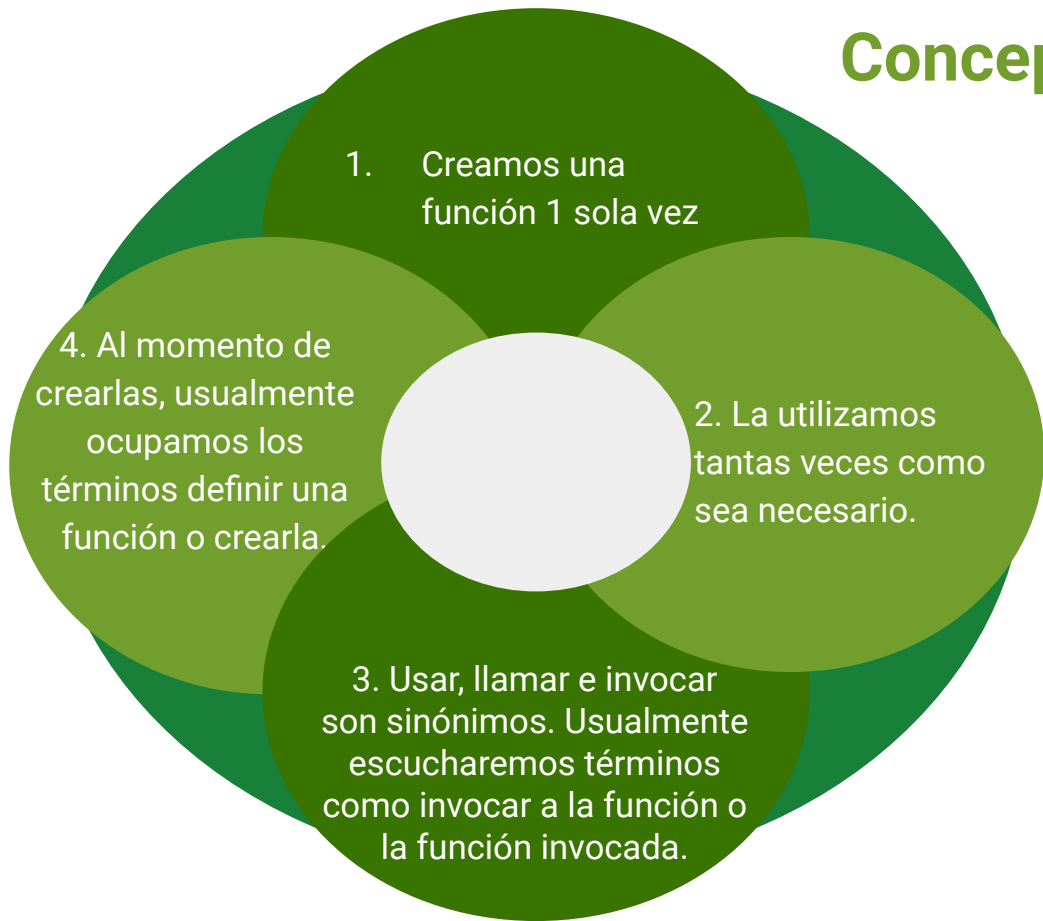
Qué es una función

- Una función es un conjunto de instrucciones diseñadas para realizar una tarea específica o calcular un valor.
- Las funciones en JavaScript son bloques de código reutilizables que pueden recibir datos de entrada (parámetros), realizar operaciones y devolver un resultado.

```
function saludar(nombre) {  
  console.log("¡Hola, " + nombre + "!");  
}
```

En este ejemplo, saludar es el nombre de la función y (nombre) es el parámetro que la función acepta. Cuando llamas a esta función y le pasas un nombre como argumento, mostrará un saludo personalizado.

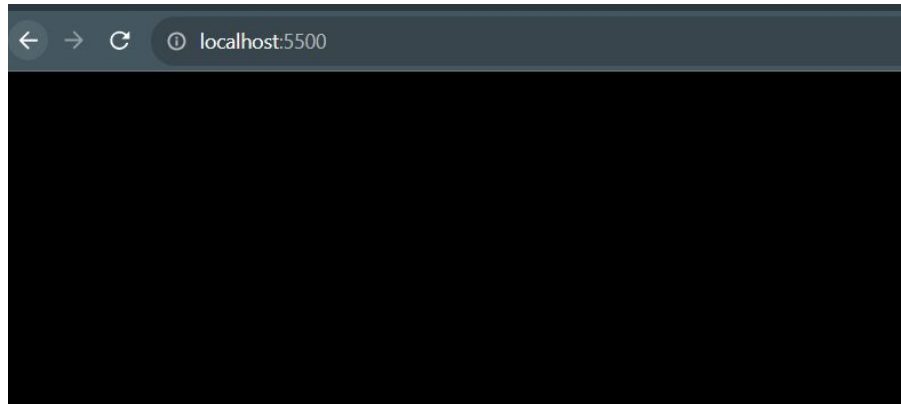
Conceptos básicos



Creando nuestra primera función

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Funciones</title>
</head>
<body>
  <script>
    function pintar_negro() {
      elemento = document.querySelector("body");
      elemento.style.backgroundColor = "black"
    }

    pintar_negro()
  </script>
</body>
</html>
```



Entendamos el código

Definición de la función

```
function pintar_negro(){  
  elemento = document.querySelector("body")  
  elemento.style.backgroundColor = color  
}  
pintar_negro();
```

Entre llaves se encuentra el **cuerpo** de la función.

El cuerpo de la función contiene toda la lógica que dicha función ejecuta.

Invocación o llamar a la función declarada.

/* Funciones y DOM*/

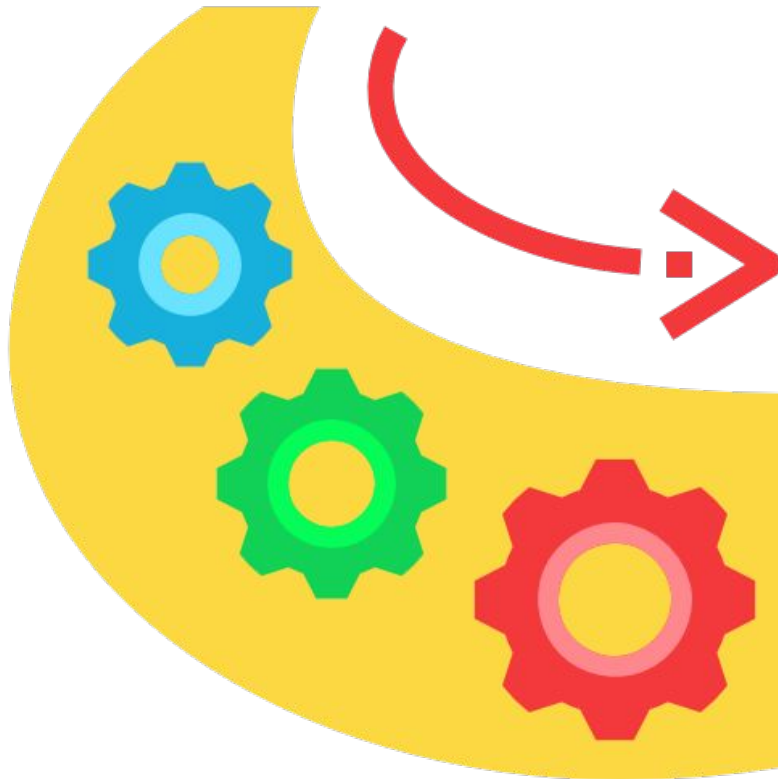
Reutilizar funciones

La reutilización de funciones es una práctica fundamental en programación y ofrece varios beneficios. Aquí hay algunas situaciones comunes en las que se reutilizan funciones en JavaScript:

1. Ejecución Repetitiva de Código
2. Manejo de Eventos
3. Código Modular y Organizado
4. Abstracción y Encapsulamiento
5. Reutilización de Bibliotecas o Módulos:

Llamar funciones desde el onclick de un elemento

Llamar funciones desde el evento onclick de un elemento en el DOM (Document Object Model) es una forma común de ejecutar código JavaScript en respuesta a la interacción del usuario.



¿Cómo lo podemos hacer?

Dentro de onclick llamaremos a la función, la función debe estar definida dentro de script. El script puede estar en el mismo archivo o en otro.

```
<button onclick="pintar('black')"> Pintar negro</button>
<script>
  function pintar(color) {
    elemento = document.querySelector("body")
    elemento.style.backgroundColor = color
  }
</script>
```

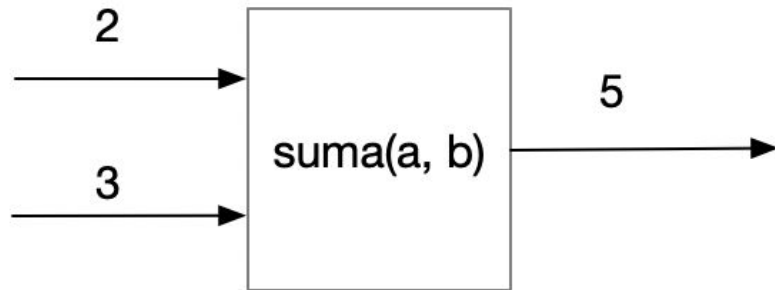
¿Cómo lo podemos hacer?

Ahora que onclick llama a la función, se hace sencillo reutilizar la lógica.

```
<button onclick="pintar('black')"> Pintar negro</button>
<button onclick="pintar('red')"> Pintar rojo</button>
<button onclick="pintar('green')"> Pintar verde</button>
<script>
  function pintar(color) {
    elemento = document.querySelector("body");
    elemento.style.backgroundColor = color;
  }
</script>
```


/* Retorno de las funciones */

Valor y funciones



Las funciones reciben valores y pueden además devolver valores, esto se debe a que pueden ser tratadas como valores en sí mismas.

```
function suma(a, b){  
  return a + b  
}
```

```
alert(suma(2,3)) /* Opción 1 */  
resultado = suma(2,3) /* Opción 2 */  
console.log(resultado)
```

Devolver el valor de funciones

```
» function suma (a, b){  
  alert (a + b)  
}  
suma(2, 3)  
← undefined
```

```
function suma (a, b) {  
  return a + b  
}  
suma(2, 2)
```

Sin embargo, no todas las funciones devuelven un valor, sino solo aquellas que tienen return.

Ejercicio: Priorizando tareas

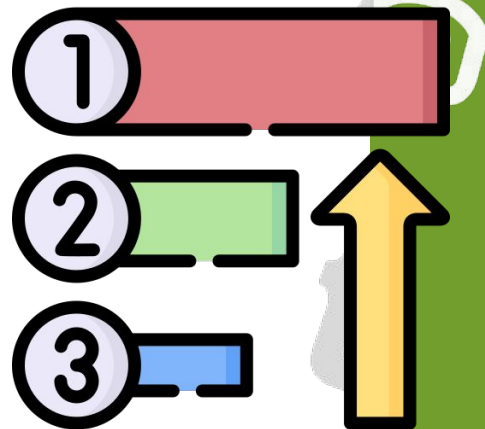


Contexto

Eres un desarrollador trabajando en una aplicación de seguimiento de tareas. Te piden desarrollar una función `getPrioridad` que obtenga la prioridad de una tarea a partir de su ID y devuelva un texto que indique la importancia de la tarea. La importancia se determina según ciertos criterios establecidos por la empresa.

Los criterios son los siguientes:

- Tareas con prioridad mayor a 7 se consideran de "Alta Importancia".
- Tareas con prioridad igual a 7 se consideran de "Importancia Exacta".
- Tareas con prioridad menor a 7 se consideran de "Baja Importancia".



Revisemos el Desafío: "Funciones"



Recomendaciones

- Para los requerimientos 1 y 2 recuerda que tienes material asociado en la “Guía - Funciones” en el capítulo de Arrow functions.
- Revisa en la “Guía - Funciones” las funciones anónimas.
- El siguiente script te ayudará a realizar acciones en función de que se presione una tecla, además en la guía de estudio encontrarás un ejemplo.

```
document.addEventListener('keydown', function (event) {  
  if (event.key === 'a') {  
    /* Cambiar a color 1 */  
  } else if (event.key === 's') {  
    /* Cambiar a color 2 */  
  }  
})
```

{desafío}
latam_

*Academia de
talentos digitales*

